

Exploring Object-Oriented Principles in Deep Learning Frameworks: PyTorch vs. TensorFlow

Yutong Zhang, Xujia Zhang

Abstract

All code and writeup is stored in this repo
https://github.com/XUJIAZHANG2002/CSCI5448_project

Introduction

- What are PyTorch and TensorFlow?
- Why compare their object-oriented (OO) design?

Background: Object-Oriented Principles

- Discuss key OO principles: Encapsulation, Inheritance, Polymorphism, Composition and Abstraction.
- Briefly introduce key design patterns: Factory, Template, Strategy & Observer.

Object-Oriented Design in PyTorch

- Explain the `nn.Module` hierarchy and its role in model composition.
 - Inheritance and Polymorphism: all layers are child class of `nn.Module`.
 - Composition: user's network is also subclass of `nn.Module` and compose multiple layers of `nn.Module`.
- Describe optimizers hierarchy and relation to Strategy pattern objects.
 - Optimizer plays a big role in training neural network.
 - Inheritance and Polymorphism: all optimizer implementations are child class of `optim.Optimizer`.
 - User compose trainable routine with different optimizer, kind of like Strategy pattern.
- Provide a small, runnable code example demonstrating these principles.
 - FullyConnected, CNN, Attention/Transformer.

Object-Oriented Design in TensorFlow / Keras

- Analyze `tf.keras.Model` and `Layer` as OO constructs.
- Discuss callbacks as an implementation of the Observer pattern.
- Explain optimizers and loss functions as interchangeable Strategy objects.
- Provide equivalent TensorFlow/Keras code examples.

Comparison and Discussion

- Identify similarities: shared OO foundations.
- Highlight differences: PyTorch's dynamic modular design vs. TensorFlow's layered abstractions.
- Discuss implications for maintainability, extensibility, and testability.

Conclusion

- Summarize why OO principles matter in modern ML frameworks.
- Discuss the broader impact on developer experience and software engineering practices.